

COMPUTER SCIENCE 229: DEEP LEARNING

Lecture Notes • Neural Networks & Backpropagation Fundamentals

Lecture Overview

This lecture covers the mathematical formulation of artificial neural networks, multilayer perceptrons (MLP), forward propagation, activation functions (ReLU, Sigmoid, Softmax), and gradient computation using the chain rule via Backpropagation. It highlights common optimization challenges including vanishing and exploding gradients and learning rate schedules.

Key Definitions & Core Terminology

- **Multilayer Perceptron (MLP):** A feedforward artificial neural network consisting of input, hidden, and output layers of non-linearly activating nodes.
- **Backpropagation:** An efficient algorithmic method for computing gradients of the loss function with respect to weights using recursive application of the calculus chain rule.
- **Activation Function:** A non-linear mathematical operation applied to neuron pre-activations enabling deep networks to approximate continuous functions.
- **Vanishing Gradient Problem:** A training roadblock where gradients decay exponentially across layers when saturating activations like Sigmoid are used in deep architectures.

Core Topics & Analytical Insights

1. Mechanics of Forward and Backward Passes

- In the forward pass, input vectors are successively mapped: $Z^l[l] = W^l[l] * A^l[l-1] + b^l[l]$, followed by activation $A^l[l] = g(Z^l[l])$.
- Activations are cached in memory during forward propagation because they are required to calculate partial derivatives during the backward pass.
- Backpropagation computes error delta vectors recursively from the final loss L back to the first layer, achieving $O(W)$ computational complexity.

2. Activation Functions: ReLU vs Sigmoid

- Sigmoid saturates for high absolute values ($|z| > 4$), driving derivatives close to zero and causing severe vanishing gradients.
- ReLU (Rectified Linear Unit) $f(z) = \max(0, z)$ provides a constant unit gradient of 1 for all positive inputs, enabling stable training of very deep networks.
- Leaky ReLU addresses the dying neuron problem by assigning a small non-zero slope (e.g. 0.01) to negative values.

Important Formulas & Quantitative Facts

Layer Pre-activation: $Z^l[l] = W^l[l] * A^l[l-1] + b^l[l]$

Affine linear transformation of previous activations with weight matrix W and bias b .

Error Delta Chain Rule: $\delta^l[l] = (W^{l+1}[l]^T * \delta^{l+1}[l]) \text{ (hadamard) } g'^l[l](Z^l[l])$

Propagation of error backwards weighted by transposed matrix and activation derivative.

Weight Update: $W^l[l] := W^l[l] - \alpha * (\delta^l[l] * (A^l[l-1])^T)$

Stochastic gradient descent parameter update with learning rate α .